CPSC 335 – Project 2

Name: Chase Hales
Email: [your CSUF email]@csu.fullerton.edu
Course: CPSC 335 Algorithm Engineering, Fall 2026
Submission: Project 2
Language: Python 3

Problem 1: Counting Reverse Pairs (Divide and Conquer)

A reverse pair is a pair (i, j) with i < j and nums[i] > 2 × nums[j]. The algorithm modifies merge sort. Each call splits the array in half, recursively counts and sorts each half, counts the pairs that cross the halves with a single two-pointer pass, then merges the halves.

Pseudocode

```
function COUNT-REVERSE-PAIRS(nums)
    (sorted, count) ← SORT-AND-COUNT(nums)
    return count

function SORT-AND-COUNT(A[0..n-1])
    if n ≤ 1 then
        return (A, 0)                          // base case: no pairs
    mid ← ⌊n / 2⌋
    (L, leftCount)  ← SORT-AND-COUNT(A[0..mid-1])   // divide + conquer
    (R, rightCount) ← SORT-AND-COUNT(A[mid..n-1])

    // combine step 1: count cross pairs (L and R are sorted ascending)
    crossCount ← 0
    j ← 0
    for i ← 0 to |L| - 1 do
        while j < |R| and L[i] > 2 × R[j] do
            j ← j + 1
        crossCount ← crossCount + j         // R[0..j-1] all pair with L[i]

    // combine step 2: merge the two sorted halves
    M ← empty list;  a ← 0;  b ← 0
    while a < |L| and b < |R| do
        if L[a] ≤ R[b] then append L[a] to M; a ← a + 1
        else                append R[b] to M; b ← b + 1
    append L[a..|L|-1] and R[b..|R|-1] to M

    return (M, leftCount + rightCount + crossCount)
```

Why the two-pointer pass is correct: because L is sorted ascending, if $L[i] > 2 \times R[j]$ then $L[i+1] \geq L[i] > 2 \times R[j]$ as well. So j never has to move backward. Every element left of j forms a pair with L[i], and the pass counts every cross pair exactly once. Every pair (i, j) with $i < j$ either lies entirely in one half, where the recursion counts it, or crosses the halves, where this pass counts it. Sorting the halves does not change which elements are in the left half and which are in the right half, so the cross count stays valid.

Baseline (not the graded solution): checking every pair takes $n(n-1)/2$ comparisons, which is $O(n^2)$. The program runs it only to double-check its answers.

Mathematical Analysis

Step count of the combine work. For a subarray of size n:

- Splitting (copying the two halves): n steps.
- Cross-pair pass: the outer loop runs n/2 times, and j increases at most n/2 times in total, so at most n steps.
- Merge: each step outputs one element, so n steps.
- Total combine work: $f(n) \leq 3cn$ for a constant c, so $f(n) = \Theta(n)$.

Recurrence relation. Each call makes two recursive calls on halves of the array plus linear combine work:

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1)$$

Master Theorem. The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 2$, $b = 2$, and $f(n) = \Theta(n)$. Then $n^{\log_b a} = n^{\log_2 2} = n^1 = n$. Since $f(n) = \Theta(n^{\log_b a})$, **Case 2** of the Master Theorem applies, which gives $T(n) = \Theta(n^{\log_b a} \log n)$.

Efficiency class: $T(n) = \Theta(n \log n)$, so $O(n \log n)$. Master Theorem Case 2.

Extra space is O(n) for the merged arrays. The recursion depth is $\lceil \log_2 n \rceil$.

Problem 2: Counting Islands in a Grid

Each land cell ('1') is a vertex of a graph, with an edge to each land cell directly above, below, left, or right of it. An island is a connected component of this graph. The algorithm scans every cell. When it finds an unvisited '1', it counts a new island and runs a depth-first search (DFS)

from that cell to mark the entire island as visited. The DFS uses an explicit stack rather than recursion, so a large island cannot overflow the call stack.

Pseudocode

```
function COUNT-ISLANDS(grid[0..m-1][0..n-1])
    if m = 0 or n = 0 then return 0
    visited[0..m-1][0..n-1] ← false          // all cells unvisited
    islands ← 0
    for r ← 0 to m - 1 do
        for c ← 0 to n - 1 do
            if grid[r][c] = '1' and not visited[r][c] then
                islands ← islands + 1           // found a new island
                visited[r][c] ← true
                stack ← [ (r, c) ]
                while stack is not empty do       // DFS over this island
                    (row, col) ← POP(stack)
                    for each (dr, dc) in ((-1, 0), (1, 0), (0, -1), (0, 1)) do
                        nr ← row + dr;  nc ← col + dc
                        if 0 ≤ nr < m and 0 ≤ nc < n
                           and grid[nr][nc] = '1' and not visited[nr][nc] then
                            visited[nr][nc] ← true      // mark when pushed
                            PUSH(stack, (nr, nc))
    return islands
```

Mathematical Analysis (method: step counts)

Let the grid have m rows and n columns, so it has mn cells.

- **Initializing visited:** one step per cell, so mn steps.
- **Outer double loop:** runs exactly mn times. Each iteration does a constant amount of work (one test), so this part takes $c_1$mn steps.
- **DFS work:** a cell is pushed onto the stack only when it changes from unvisited to visited, and it never changes back. So each cell is pushed, and later popped, *at most once* over the entire run, and there are at most mn pops. Each pop examines exactly 4 neighbors with constant work per neighbor, so all DFS work combined takes at most $4c_2$mn steps.
- **Total:** $T(m, n) \leq mn + c_1mn + 4c_2mn = (1 + c_1 + 4c_2)\cdot mn$.

Since $1 + c_1 + 4c_2$ is a constant, $T(m, n) \in O(mn)$. The outer loop alone always performs mn iterations, so $T(m, n) \geq mn$, which gives a lower bound of $\Omega(mn)$.

Efficiency class: $T(m, n) = \Theta(mn)$, so $O(m \times n)$. Method: step counts.

Extra space is O(mn): the visited array has mn entries, and in the worst case (an all-land grid) the stack can hold O(mn) cells.

Results

Each program runs the three examples from the assignment. All six produce the expected output. Problem 1 also matches the $O(n^2)$ brute-force count on all three examples and on 2,000 additional random arrays tested during development.

Problem 1 output

```
Problem 1: Counting Reverse Pairs (divide and conquer)
Example 1: nums = [5, 1, 8, 3, 2]
  Output: 4   (expected 4, brute force 4)  PASS
Example 2: nums = [2, 4, 3, 5, 1, 12, 6]
  Output: 3   (expected 3, brute force 3)  PASS
Example 3: nums = [9, 4, -2, 7, 1, 3, 0]
  Output: 13   (expected 13, brute force 13)  PASS
```

Problem 2 output

```
Problem 2: Counting Islands in a Grid
Example 1: 5 x 5 grid
  0 1 1 0 0
  0 1 0 0 1
  1 0 0 1 1
  1 0 1 0 0
  0 0 1 1 0
  Output: 4   (expected 4)  PASS
Example 2: 4 x 4 grid
  1 1 1 0
  1 0 1 0
  1 1 1 0
  0 0 0 1
  Output: 2   (expected 2)  PASS
Example 3: 6 x 6 grid
  1 0 1 0 1 0
  1 1 0 0 1 1
  0 0 0 1 0 0
  0 1 1 1 0 1
  0 0 0 0 0 1
  1 1 0 1 0 0
  Output: 7   (expected 7)  PASS
```